

Project 11: Projections, eigenvectors, Principal Component Analysis, and face recognition algorithms

Goals: To use eigenvalues and eigenvectors of a matrix to create a simple face recognition algorithm.

To get started:

- Download the file lab11.py and put it in your working directory¹³.
- Download the archive file database.zip and unpack it into your working directory. This will produce a folder “database”, which contains 33 image files in the format .pgm ¹⁴.

Python commands used: imread, shape, reshape, str, mean, np.uint8, np.double, np.ones, np.linalg.eig, np.diag

What you have to submit: The file lab11.py which you will modify during the lab session.

INTRODUCTION

In this project we will apply principal component analysis (PCA) to create a simple face recognition algorithm. The PCA algorithm works by identifying patterns in the data which allows one to reduce the dimension of the dataset. We will use the following steps to apply the PCA algorithm to face recognition:

¹³ The printout of the file lab11.m can be found in appendices of this book.

¹⁴ These files represent a subset of the database of faces from AT&T Laboratories Cambridge www.cl.cam.ac.uk/research/dtg/attarchive/facedatabase.html.

- Download the database of N images (in our case, $N = 30$). All images must have the same dimensions $m \times n$.
- Input the images person1.pgm-person30.pgm into Python as arrays. Reshape each of the arrays as a column vector $\mathbf{X}_i$, $i = 1, \dots, N$.
- Compute the mean of all images $\mathbf{X}_{mean} = \frac{1}{N} \sum_{i=1}^N \mathbf{X}_i$.
- Subtract the mean image from each of the vectors $\mathbf{X}_i$: $\tilde{\mathbf{X}}_i = \mathbf{X}_i - \mathbf{X}_{mean}$ and create a matrix $\mathbf{P}$, the i th column of which is equal to the vector $\tilde{\mathbf{X}}_i$. The matrix $\mathbf{P}$ has 30 columns and mn rows where m, n are the dimensions of the images.
- We are interested in the eigenvalues and eigenvectors of the matrix $\mathbf{PP}^T$. However, the matrix $\mathbf{PP}^T$ is usually very large. For instance, if our database consists of the images with dimensions $m \times n = 100 \times 100$, then the matrix $\mathbf{PP}^T$ has the dimensions 10000×10000 , and computation of eigenvalues and eigenvectors becomes unfeasible. Instead, we will use the fact that only at most $N - 1$ eigenvalues of the matrix $\mathbf{PP}^T$ are non-zero (why?), and those non-zero eigenvalues are the same as non-zero eigenvalues of the matrix $\mathbf{P}^T\mathbf{P}$ (why?). The matrix $\mathbf{P}^T\mathbf{P}$ has the dimensions $N \times N$ which are usually much smaller than the dimensions of the matrix $\mathbf{PP}^T$. For instance, if there are 30 images in the database (like in our case), then the dimensions of $\mathbf{P}^T\mathbf{P}$ are only 30×30 . Thus, it is much more efficient to solve the eigenvalue problem for the matrix $\mathbf{P}^T\mathbf{P}$.
- If $\mathbf{u}_i$ is an eigenvector of the matrix $\mathbf{P}^T\mathbf{P}$ corresponding to a non-zero eigenvalue λ_i , then $\mathbf{v}_i = \mathbf{P}\mathbf{u}_i$ is the eigenvector of the matrix $\mathbf{PP}^T$ corresponding to λ_i . The eigenvectors of the matrix $\mathbf{PP}^T$ constitute the set of components of the data with the largest variance between them (which are called principal components).
- Compute the projections of your sample data onto the linear subspace generated by the vectors $\mathbf{v}_i$. The power of the algorithm is in the dimension reduction. Our initial images of faces “live” in the mn -dimensional real space. For instance, if the images are 100×100 pixels, then the dimension of the space is equal to $100^2 = 10000$. If we would try to compare these images to each other pixel by pixel, we would need to compare 10000 pixels. However, not all of these pixels are significant. Principal Component Analysis allows us to identify the important information about the image and compare only that information. For instance, if we are working with a database of 30 images, then we will need to compare only 29 numbers. That is a dimensional reduction from 10000 to only 29! The PCA algorithm is a work horse behind many modern techniques in data mining, including face recognition algorithms.

TASKS

1. Open the file lab11.m and run the first code cell titled # Input database files into the Python:

This code will input the database images into Python, reshape each image from the rectangular array to a column vector, and create the matrix P. Observe the use of the string function num2str in the imread command and the use of the reshape function to change the dimensions of the arrays containing the image data. Instead of the rectangular arrays with the elements corresponding to the pixel colors we now have a set of long vectors where columns of each image have been “stacked” on top of each other. **Variables: Database Size, m, n, P**

```
import numpy as np
from PIL import Image
import os
import numpy as np
from PIL import Image
import os
import matplotlib.pyplot as plt

# Parameters
Database_Size = 30
database_path = 'C:/Users/Harshith/Downloads/laproject/materials/database'

# Initialize list to store image vectors
P = []

# Reading images from the database
for j in range(1, Database_Size + 1):
    image_path = os.path.join(database_path, f'person{j}.pgm')
    image = Image.open(image_path)
    image_array = np.array(image)

    # Get dimensions of the image
    m, n = image_array.shape

    # Reshape the image array to a column vector
    image_vector = image_array.reshape(m * n, 1)
    P.append(image_vector)

# Convert list to numpy array (matrix)
P = np.hstack(P)

# Print out the variables for verification
print(f"Database Size: {Database_Size}")
print(f"Image dimensions (m, n): ({m}, {n})")
print(f"P matrix shape: {P.shape}")
```

2. Compute the mean image by applying the mean function to the the matrix P row-wise.
Display the mean image by using the function `imshow`.

Variables: mean face

```
# Compute the mean face
mean_face = np.mean(P, axis=1)

# Reshape the mean face back to the original image dimensions
mean_face_image = mean_face.reshape(m, n)

# Display the mean face image
plt.imshow(mean_face_image, cmap='gray')
plt.title('Mean Face')
plt.axis('off') # Hide axis
plt.show()
```

3. Subtract the mean of the images from each of the columns of the matrix P :
Notice the conversion to the double type of the array P .

```
# Compute the mean face
mean_face = np.mean(P, axis=1)

# Convert P to double (float64 in numpy)
P = P.astype(np.float64)

# Subtract the mean face from each column of P
mean_face_column = mean_face.reshape(-1, 1)
P = P - mean_face_column @ np.ones((1, Database_Size))

# Print the first column of P to verify subtraction
print(P[:, 0])
```

4. Compute the covariance matrix $\mathbf{P}^T \mathbf{P}$ and compute the eigenvalues and the eigenvectors of this matrix $\mathbf{P}^T \mathbf{P}$ by using the built in function `eig`. The function `eig` returns two arrays, one of which has eigenvalues on the main diagonal, and the other has eigenvectors as columns. Observe that one of the eigenvalues is very close to zero.

Variables: Vectors, Values, EigenVectors

Q1: Why does the matrix $\mathbf{P}^T \mathbf{P}$ have a zero eigenvalue?

```

# Compute the covariance matrix  $P^T * P$ 
PTP = P.T @ P

# Compute the eigenvalues and eigenvectors of  $P^T * P$ 
Values, Vectors = np.linalg.eig(PTP)

# Compute the actual eigenvectors of the covariance matrix
EigenVectors = P @ Vectors

# Normalize the eigenvectors
EigenVectors = EigenVectors / np.linalg.norm(EigenVectors, axis=0)

# Display the first few eigenvalues for verification
print("Eigenvalues:", Values)

```

5. Display the set of eigenfaces on the same figure by running the code cell # Display the set of eigenfaces:

If you did everything correctly, you will see an image containing 29 eigenfaces, the first five of which are shown in Fig. 16. Observe that the mean face has been added back to the eigenfaces.

Variables: EigenFaces

```

# Display the set of eigenfaces
eigenfaces = []

for j in range(1, Database_Size):
    eigenface = EigenVectors[:, j] + mean_face
    eigenface_image = eigenface.reshape(m, n)
    eigenfaces.append(eigenface_image)

# Concatenate the eigenfaces horizontally
EigenFaces = np.hstack(eigenfaces)

# Display the eigenfaces
plt.figure(figsize=(15, 5))
plt.imshow(EigenFaces, cmap='gray')
plt.title('Eigenfaces')
plt.axis('off') # Hide axis
plt.show()

```

6. The matrix $\mathbf{PP}^T$ is symmetric. It follows from the theorems of linear algebra that the eigenvectors corresponding to the distinct eigenvalues are orthogonal. Write code to verify the last statement. You will need to create the matrix
- $$\text{Products} = \text{EigenVectors}' * \text{EigenVectors}$$
- and observe that it is a diagonal matrix. This matrix contains all possible dot products of the eigenvectors of the matrix $\mathbf{PP}^T$. The fact that the eigenvectors are orthogonal will be used in the following sections to find orthogonal projections.

Variables: Products

Q2: Why is the matrix $\mathbf{PP}^T$ symmetric?

```
# Compute the Products matrix
Products = EigenVectors.T @ EigenVectors

# Print the Products matrix to verify orthogonality
print("Products matrix:")
print(Products)

# Check if Products matrix is diagonal
is_diagonal = np.allclose(Products, np.diag(np.diagonal(Products)))
print(f"Is Products matrix diagonal? {is_diagonal}")
```

7. In this project we will use the obtained eigenfaces to perform several operations. Our first task will be to “unmask” an altered face (sunglasses have been digitally added to the image). The main idea behind this code is in computing the projection of the vector of data (obtained from the altered image) onto the subspace spanned by the eigenvectors. This



Figure 16: The first five eigenfaces

projection is then displayed on the screen. Now run the code cell titled # Recognition of an altered image (sunglasses):

In this code, we start by uploading the image person30altered1.pgm into Python. The image is then reshaped as a long vector similarly to the way it was done before. Then we compute

the projection of the new vector with the subtracted vector mean face onto the space spanned by the set of the eigenvectors and add the vector mean _face back. Finally, the reconstructed image is displayed. Observe that we get a very good approximation of the initial image without sunglasses.

Variables: image_read, U, NormsEigenVectors, W, U approx

```
# Define image dimensions
m, n = 112, 92

# Read the altered image
altered_image_path =
'C:/Users/Harshith/Downloads/laproject/materials/database/person32.pgm'
image_read = Image.open(altered_image_path)
image_array = np.array(image_read)
U = image_array.reshape(m * n, 1)

# Compute the norms of the eigenvectors
Products = EigenVectors.T @ EigenVectors
NormsEigenVectors = np.diag(Products)

# Compute the projection coefficients
W = EigenVectors.T @ (U.astype(np.float64) - mean_face.reshape(-1, 1))
W = W / NormsEigenVectors.reshape(-1, 1) # Ensure proper division

# Reconstruct the image from the projection
U_approx = EigenVectors @ W + mean_face.reshape(-1, 1)

# Print shapes for debugging
print("Shape of U_approx:", U_approx.shape)
print("Expected shape:", (m * n, 1))

# Ensure the shape matches for reshaping
if U_approx.shape[0] == m * n and U_approx.shape[1] == 1:
    image_approx = U_approx.reshape(m, n).astype(np.uint8)
else:
    raise ValueError(f"Cannot reshape array of size {U_approx.size} into shape ({m}, {n})")

# Display the original altered image and the reconstructed image
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image_array, cmap='gray')
plt.title('Original Altered Image')
```

```
plt.axis('off') # Hide axis

plt.subplot(1, 2, 2)
plt.imshow(image_approx, cmap='gray')
plt.title('Reconstructed Image')
plt.axis('off') # Hide axis

plt.show()
```

8. Modify the code above to unmask the image person30altered2.pgm.

```
# Define image dimensions
m, n = 112, 92

# Read the altered image
altered_image_path = 'C:/Users/Harshith/Downloads/laproject/materials/database/person33.pgm'
image_read = Image.open(altered_image_path)
image_array = np.array(image_read)
U = image_array.reshape(m * n, 1)

# Compute the norms of the eigenvectors
Products = EigenVectors.T @ EigenVectors
NormsEigenVectors = np.diag(Products)

# Compute the projection coefficients
W = EigenVectors.T @ (U.astype(np.float64) - mean_face.reshape(-1, 1))
W = W / NormsEigenVectors.reshape(-1, 1) # Ensure proper division

# Reconstruct the image from the projection
U_approx = EigenVectors @ W + mean_face.reshape(-1, 1)

# Print shapes for debugging
print("Shape of U_approx:", U_approx.shape)
print("Expected shape:", (m * n, 1))

# Ensure the shape matches for reshaping
if U_approx.shape[0] == m * n and U_approx.shape[1] == 1:
    image_approx = U_approx.reshape(m, n).astype(np.uint8)
else:
    raise ValueError(f"Cannot reshape array of size {U_approx.size} into shape ({m}, {n})")

# Display the original altered image and the reconstructed image
```

```

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image_array, cmap='gray')
plt.title('Original Altered Image')
plt.axis('off') # Hide axis

plt.subplot(1, 2, 2)
plt.imshow(image_approx, cmap='gray')
plt.title('Reconstructed Image')
plt.axis('off') # Hide axis

plt.show()

```

9. Now let us compute the approximation of the face which is not in the database. To do this, modify the code above again to approximate the image person31.pgm. Observe, that the results of the last two approximations are somewhat worse than the very first one. This can be explained by the fact that there is an insufficient number of images in our database which do not span a wide enough variety of faces.

```

# # Recognition and approximation of a new face (person31.pgm)
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

# Define image dimensions
m, n = 112, 92

# Read the new image (person31.pgm)
new_image_path = 'C:/Users/Harshith/Downloads/laproject/materials/database/person31.pgm'
image_read = Image.open(new_image_path)
image_array = np.array(image_read)
U = image_array.reshape(m * n, 1)

# Compute the norms of the eigenvectors
Products = EigenVectors.T @ EigenVectors
NormsEigenVectors = np.diag(Products)

# Compute the projection coefficients
W = EigenVectors.T @ (U.astype(np.float64) - mean_face.reshape(-1, 1))
W = W / NormsEigenVectors.reshape(-1, 1) # Ensure proper division

# Reconstruct the image from the projection
U_approx = EigenVectors @ W + mean_face.reshape(-1, 1)

```

```

# Print shapes for debugging
print("Shape of U_approx:", U_approx.shape)
print("Expected shape:", (m * n, 1))

# Ensure the shape matches for reshaping
if U_approx.shape[0] == m * n and U_approx.shape[1] == 1:
    image_approx = U_approx.reshape(m, n).astype(np.uint8)
else:
    raise ValueError(f"Cannot reshape array of size {U_approx.size} into shape ({m}, {n})")

# Display the original new image and the reconstructed image
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image_array, cmap='gray')
plt.title('Original New Image')
plt.axis('off') # Hide axis

plt.subplot(1, 2, 2)
plt.imshow(image_approx, cmap='gray')
plt.title('Reconstructed Image')
plt.axis('off') # Hide axis

plt.show()

# Variables: image_read, U, NormsEigenVectors, W, U_approx

```

Project 12: Matrix eigenvalues and the Google's PageRank algorithm

Goals: To apply matrix eigenvalues and eigenvectors to ranking of webpages in the World Wide Web.

To get started:

- Download the file lab12.py and put it in your working directory¹⁵.

¹⁵ The printout of the file lab12.m is given in the appendices of this book.